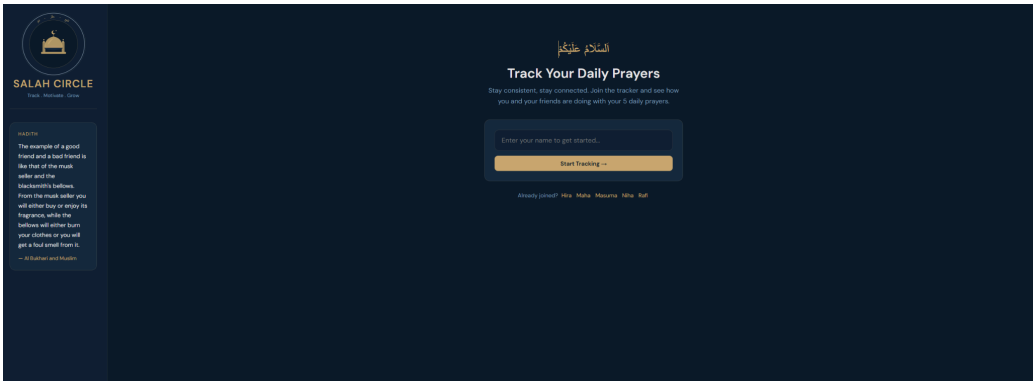
Salah Circle: A Prayer Tracking Web Application

Md Rafi | Cloud Computing | Professor Zhou Ji

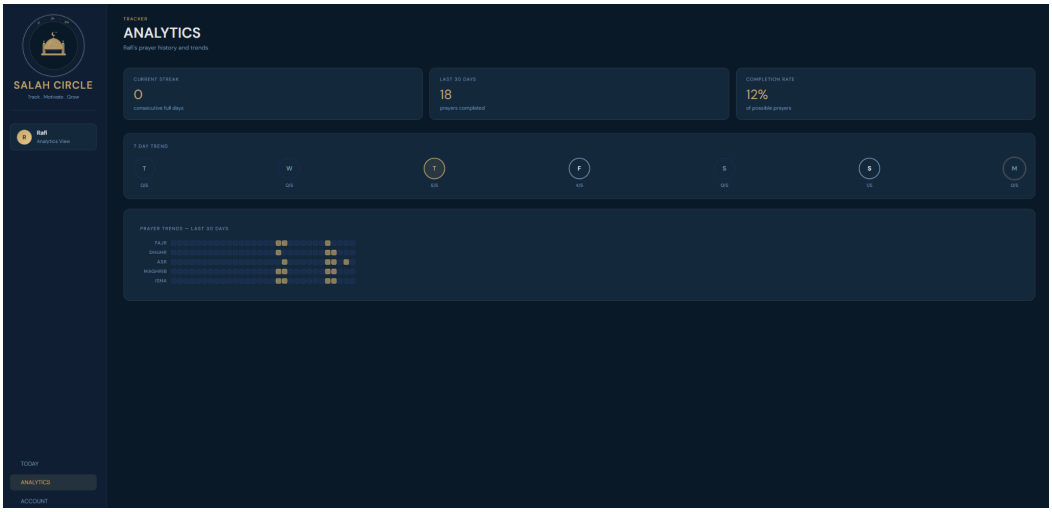
Project Overview

Salah Circle is a full-stack web application designed to help Muslims track their five daily prayers. The application allows multiple users to log their prayers, view each other's progress in a social feed, analyze their prayer history through an analytics dashboard, and receive personalized motivational messages powered by Google Gemini AI. The app also displays real-time prayer times fetched from the Aladhan API. One of the main reasons I made this app is so that one friend can motivate other friends to pray if they miss and keep their friends accountable.

This project was built as an extension of Homework 4, which was a simple single-user to-do list tracker. Salah Circle takes the same foundational architecture. A Flask backend API, a Flask frontend, and a SQLite database deployed on Google Cloud expands it in scope, functionality, and complexity.







Live Application URLs:

- Frontend App: http://34.42.158.172:5002
- Backend API: http://34.42.158.172:5001
- Docker Hub (API): https://hub.docker.com/r/rafi4632/prayer-tracker-api
- Docker Hub (App): https://hub.docker.com/r/rafi4632/prayer-tracker-app
- Kubernetes: http://34.45.90.149

Understanding the Setup and Components in Homework 4

Todo List Application
Homework 4 was deployed using a two-server architecture on Google Cloud:

- Backend (api.py): A Flask REST API running on a Google Cloud VM on port 5001. It exposed a single endpoint (/api/items) to handle GET, POST, PUT, and DELETE operations for todo items. Data was stored in a SQLite database (todolist.db).
- Frontend (todolist.py): A Flask web application running on port 5002 that rendered HTML templates and communicated with the backend API via HTTP requests.
- Database: SQLite, a lightweight, file-based relational database stored on the VM.
- Containerization: The frontend was containerized with Docker using a simple Dockerfile based on a Python base image.
- Deployment: The Docker image was pushed to Docker Hub and deployed to a Google Kubernetes Engine (GKE) cluster using kubectl. A LoadBalancer service exposed the app publicly at http://35.188.204.64.

The architecture of Homework #4 was:
User (Browser) → Kubernetes (Frontend) → Cloud VM (API) → SQLite (Database)

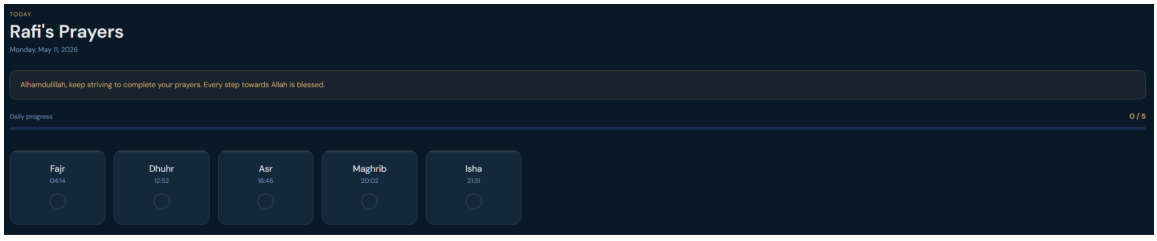
The application had four basic features: adding tasks, marking tasks complete, deleting tasks, and viewing all tasks. There was no user authentication, no analytics, no external APIs, and no AI features.

The final project, Salah Circle, was built on the same foundational architecture as Homework 4, but extended it significantly across every dimension

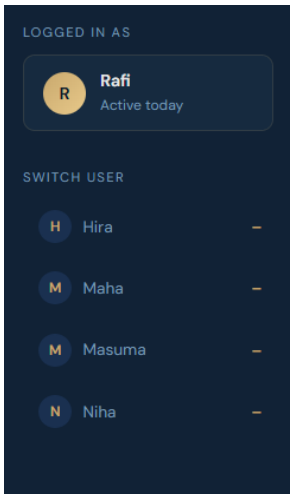
Todo List	Salah Circle
Single user, no accounts	Multi-user with session management
One endpoint: /api/items	8 REST API endpoints
No external APIs	Aladhan API + Google Gemini AI API
No analytics	30-day heatmap, 7-day trend, streak tracking
No social features	Social feed showing all users' prayer progress
Basic HTML styling	Full custom Islamic gold-theme UI
No AI features	AI-powered motivational messages
One Docker image	Two Docker images (api + app) with Docker Compose

Application Features

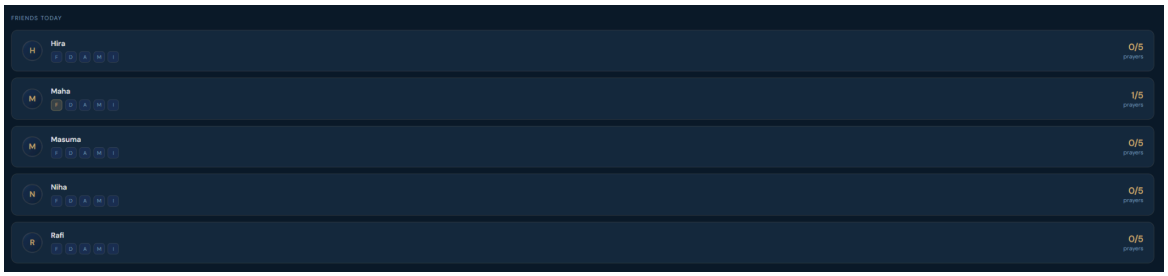
- Prayer Tracking
  The core feature of the application is the ability for users to log their five daily prayers: Fajr, Dhuhr, Asr, Maghrib, and Isha. Each prayer is displayed as a card on the dashboard. Users can toggle each prayer as completed or incomplete with a single click. The prayer cards display the actual prayer time for the day, fetched from the Aladhan API.



- Multi-User Support
  Unlike Homework 4, which had no user concept, Salah Circle supports multiple users. Users can join the tracker by entering their name, and the app uses Flask sessions to keep track of the currently logged-in user. Users can switch between accounts, and each user has a delete option.



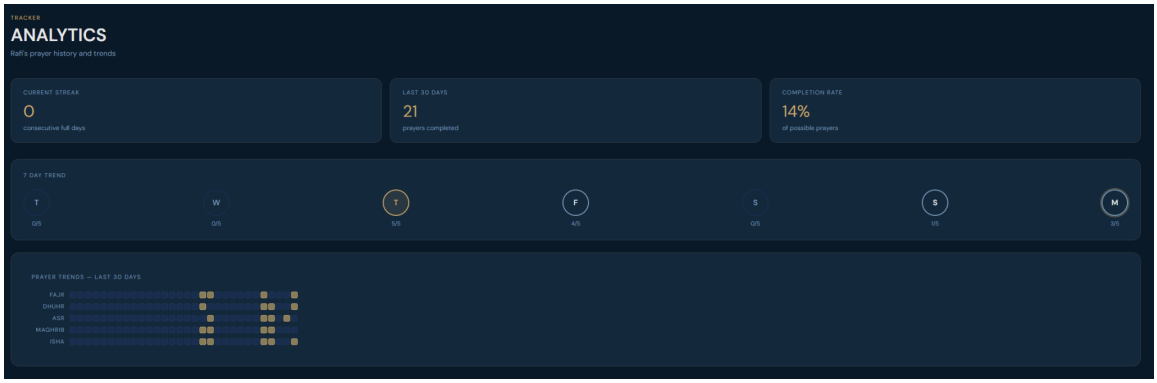
- Social Feed
  The social feed section on the main dashboard displays all users and their prayer completion status for the day. Each user's card shows which of the five prayers they have completed, using color-coded dots for each prayer. This creates a sense of community accountability.



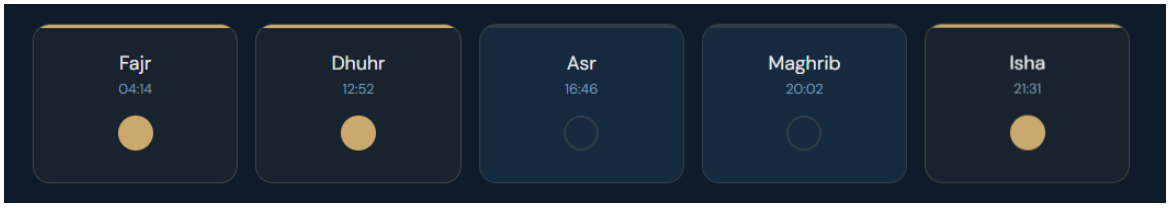
- AI-Powered Motivation
  One of the most unique features of the application is the integration of Google Gemini AI to generate personalized Islamic motivational messages. Each time a user loads the dashboard, the app calls the Gemini API with a prompt that includes which prayers the user has completed and which they have missed.



- Analytics Dashboard
  The analytics page provides users with a detailed view of their prayer history:
  - 30-Day Heatmap: A visual grid showing prayer completion for each of the five prayers over the last 30 days.
  - 7-Day Trend: Circle indicators showing how many prayers were completed each day over the past week.
  - Streak Counter: Tracks how many consecutive full days (all 5 prayers completed) the user has achieved.
  - Completion Rate: Shows the percentage of possible prayers completed over the last 30 days.



- Real-Time Prayer Times
  The application fetches real-time Islamic prayer times using the Aladhan API (api.aladhan.com). The prayer times are displayed on each prayer card on the dashboard, showing users the actual scheduled times for each prayer in their city.



- Islamic Motivation Display
  When a user is not logged in, the sidebar displays an authentic hadith from Al-Bukhari and Muslim on the importance of good companions, which aligns with the application's social and community aspects.

External APIs Used

Aladhan Prayer Times API

The Aladhan API (api.aladhan.com/v1/timingsByCity) was integrated into app.py to fetch real-time Islamic prayer times. The API accepts a city name, country, and calculation method, and returns the exact times for all five prayers. The application uses the ISNA calculation method for New York, US. The prayer times are fetched on every page load and displayed on the prayer cards.

```
HTML
GET http://api.aladhan.com/v1/timingsByCity/{date}?city=New York&country=US&method=2
```

Google Gemini AI API

The Google Gemini AI API was integrated into api.py using the Google Generative AI Python library. The integration works as follows:

- The library is installed: pip install google-generativeai
- The API key is configured: genai. configure(api_key='...')
- The model is initialized: genai.GenerativeModel('gemini-2.5-flash-lite')
- A prompt is built dynamically based on the user's completed and missed prayers for the day.
- The API returns a personalized 2-sentence motivational message using Islamic terminology.
- The message is returned to the frontend and displayed on the dashboard.

The model used is gemini-2.5-flash-lite, which is available on the free tier of the Google AI API. Error handling was implemented so that if the API quota is exceeded or the call fails, a fallback motivational message is displayed instead of crashing the application.
The initial Gemini API integration used the gemini-pro model, which Google had since deprecated. This caused silent failures where the motivation message would not appear on the dashboard. The model was updated to gemini-2.5-flash-lite, which is the current free-tier compatible model. Additionally, the free tier daily quota of 20 requests was quickly exhausted during testing, requiring careful management of API calls.

Code Implementation

Backend API (api.py)

The backend is running on port 5001. It provides the following REST API endpoints:

- GET /api/users — Returns all registered users

- POST /api/users — Creates a new user
- GET /api/prayers/today — Returns today's prayer status for a user
- POST /api/prayers/toggle — Toggles a prayer as completed or incomplete
- GET /api/social/today — Returns all users' prayer status for today
- GET /api/analytics/ — Returns heatmap, trend, and streak data
- GET /api/motivation/ — Calls Gemini AI and returns motivational message
- DELETE /api/users/ — Deletes a user and their prayer data

Frontend App (app.py)
The frontend is running on port 5002. It renders Jinja2 HTML templates and communicates with the backend API via HTTP requests using the requests library.
It manages user sessions using Flask's object. The following routes are defined:

- GET / — Renders the main dashboard
- POST /join — Creates or retrieves a user and sets the session
- GET /switch/ — Switches to a different user account
- GET /toggle/ — Toggles a prayer and redirects back
- GET /analytics — Renders the analytics page
- GET /logout — Clears the session
- GET /delete-user/ — Deletes a user

Database Schema

The application uses SQLite (prayer.db) with two tables:

```
HTML
users: id, name, created_at
prayers: id, user_id (FK), prayer_name, prayer_date, completed, completed_at
```

A UNIQUE constraint on (user_id, prayer_name, prayer_date) ensures no duplicate prayer entries. The INSERT OR IGNORE pattern is used to efficiently create prayer rows for each day without duplication.

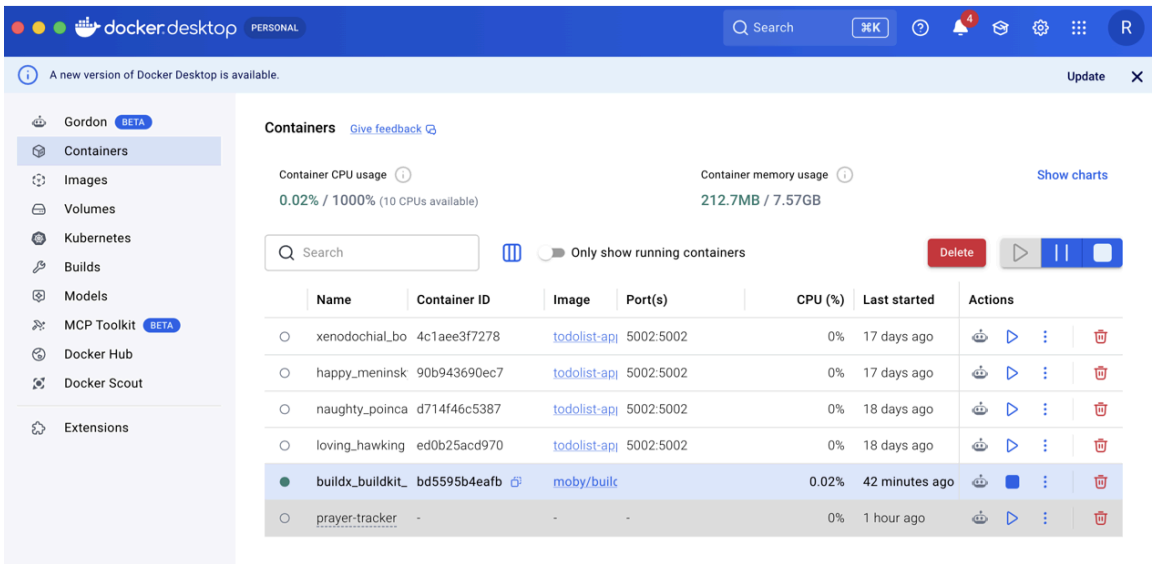

```
{
  "created_at": "2026-04-29",
  "id": 2,
  "name": "Hira"
},
{
  "created_at": "2026-04-29",
  "id": 3,
  "name": "Maha"
},
```

Deployment and Infrastructure

- Google Cloud VM
  The application is deployed on a Google Cloud Compute Engine VM instance (homework4-api-server) in the us-central1-f zone. Both api.py and app.py run directly on the VM as Python processes. Firewall rules were configured to open ports 5001 and 5002 for public access.

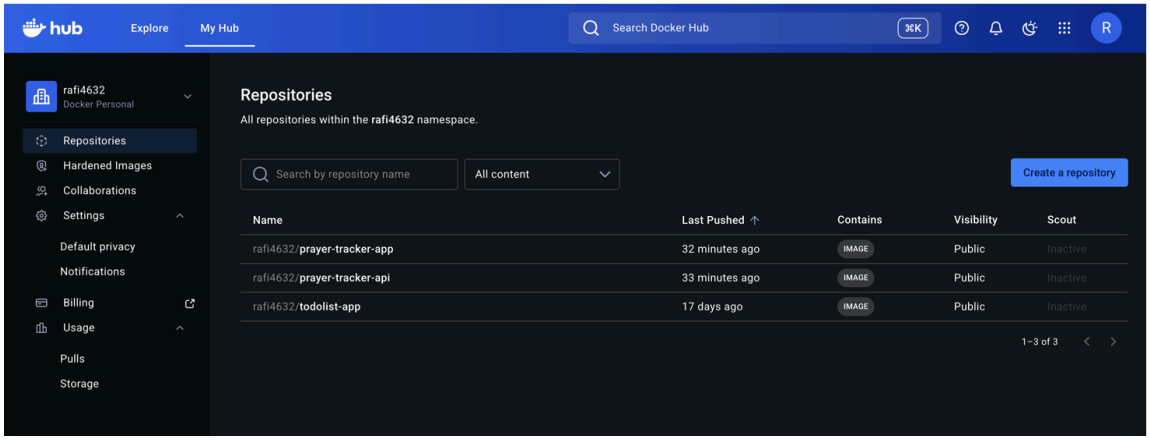


- Docker and Docker Compose
  Both the API and the frontend app were containerized using Docker. Separate Dockerfiles were created for each service (Dockerfile.api and Dockerfile.app). A docker-compose.yml file was created to orchestrate both containers, with the frontend configured to communicate with the API container via Docker's internal networking (API_URL=http://api:5001).



The Docker images were successfully built and tested locally on a Mac, and the application was verified to run at http://localhost:5002 using docker-compose up.
docker-compose up --build

- Docker Hub
  Both Docker images were pushed to Docker Hub under the rafi4632 namespace:
  - rafi4632/prayer-tracker-api:latest
  - rafi4632/prayer-tracker-app:latest



- Kubernetes (GKE)
  Kubernetes deployment was attempted using the existing todo-cluster GKE cluster from Homework 4. Deployment YAML files were created for both the API and the frontend app, including Kubernetes Deployment and Service definitions.

```
mrafi2@cloudshell:~ (cischw4)$ kubectl get pods
NAME                                  READY   STATUS             RESTARTS   AGE
prayer-tracker-api-5f6ffd7949-t2bnh   0/1     ImagePullBackOff   0          7m38s
prayer-tracker-app-58768f49c4-wmkvh   0/1     ImagePullBackOff   0          7m37s
todolist-7b5d75f585-b29qc             1/1     Running            0          176m
mrafi2@cloudshell:~ (cischw4)$ kubectl get services
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP     PORT(S)          AGE
kubernetes          ClusterIP      34.118.224.1     <none>          443/TCP          16d
prayer-tracker-api  ClusterIP      34.118.234.43    <none>          5001/TCP         8m7s
prayer-tracker-app  LoadBalancer   34.118.237.137   34.45.90.149    80:31411/TCP     8m6s
todolist            LoadBalancer   34.118.231.94    35.188.204.64   80:32207/TCP     16d
```

The frontend service was exposed using a LoadBalancer type, and an external IP 34.45.90.149 was successfully assigned.

A platform-compatibility issue was encountered during deployment. The Docker images built on a Mac M-series (ARM architecture) contained attestation manifests that caused the GKE nodes (running AMD64/Linux) to fail with an 'no match for platform in manifest' error. Multiple approaches were attempted to resolve this. This is a known challenge when developing on Apple Silicon Macs and deploying to Linux-based Kubernetes clusters. The issue is documented and understood, and represents a real-world deployment challenge that was thoroughly investigated during this project.

Data Persistence

The application uses SQLite as its database, stored as a file (prayer.db) on the server. The database was designed with the following persistence features:

- UNIQUE constraints prevent duplicate prayer entries for the same user on the same day.
- Foreign key relationships link prayers to users, ensuring data integrity.
- The INSERT OR IGNORE pattern efficiently handles the creation of daily prayer records without errors.
- Date-based queries allow historical data retrieval for the analytics dashboard.

```
HTML
volumes: ./prayer.db:/app/prayer.db
```

Conclusion

Salah Circle extends the Homework 4 Todo List application. While Homework 4 demonstrated the basic mechanics of deploying a two-server Flask application on Google Cloud with Docker and Kubernetes, the final project explores a much richer set of technologies and features.

The project successfully integrates two external public APIs, implements a multi-user social platform with analytics, uses a properly normalized relational database with two tables and foreign key constraints, containerizes both services using Docker with Docker Compose orchestration, pushes images to Docker Hub, and deploys the application live on Google Cloud, accessible via a public IP address. Overall, this project demonstrates a clear progression from the foundational cloud deployment skills learned in this course.